Discover Applied Sciences

Research

Breaking language barriers with image detection and natural language processing model for English to Spanish translation

Bakhita Salman[1] · **Andres Lopez**[1] · **Nathanielle Delapena**[1]

Received: 8 January 2025 / Accepted: 7 April 2025
Published online: 01 July 2025

Abstract
This research proposes an advanced approach that integrates multiple image detection techniques and natural language processing (NLP) methodologies for English-to-Spanish language translation. The developed software accepts an image as input, which undergoes preprocessing using adaptive thresholding, morphological transformations, and edge detection algorithms such as Canny and Sobel operators to enhance text clarity. Text detection and localization are achieved using the EfficientDet and EAST (Efficient and Accurate Scene Text) detector frameworks, followed by Optical Character Recognition (OCR) using PyTesseract, a wrapper for Google's Tesseract OCR. The detected text is passed to an NLP system for translation, which employs a sequence-to-sequence transformer model implemented with Keras, TensorFlow, and NumPy. Additional techniques, such as Byte Pair Encoding (BPE) for text tokenization and positional encoding for transformer-based attention, improve translation efficiency. An English-Spanish dictionary from Anki and a large parallel corpus dataset were used for training. The NLP pipeline leverages semantic analysis, part-of-speech tagging, and dependency parsing to preserve grammatical structure and context. Fine-tuning the transformer model parameters, including learning rate scheduling and gradient clipping, further optimized system performance. The research demonstrates a 93.7% translation accuracy, achieved by combining state-of-the-art image processing algorithms, advanced transformer architectures, and a robust training dataset. This hybrid approach significantly improves the accuracy of English-to-Spanish translations, validating the effectiveness of integrating computer vision and NLP technologies.

Article highlights

- Enhanced translation accuracy—AI-powered translation models improve English-to-Spanish accuracy by preserving grammar, sentence structure, and contextual meaning. The integration of deep learning and NLP ensures precise translations across various text types.
- Optimized image-to-text extraction—advanced OCR techniques, including adaptive thresholding, morphological transformations, and deep learning-based text detection, enhance the accuracy of extracting text from images, even in complex backgrounds or poor lighting conditions.
- Scalable AI solutions for real-world applications—the combination of computer vision and NLP enables practical applications in multilingual communication, document translation, accessibility for visually impaired users, and real-time text recognition for travel, business, and education. The AI-driven approach ensures scalability across diverse environments and languages.

✉ Bakhita Salman, bakhita.salman@tamiu.edu; Andres Lopez, aivan_lopez@dusty.tamiu.edu; Nathanielle Delapena, nathanielledelapena@dusty.tamiu.edu | [1]School of Engineering, Texas A&M International University, Laredo, USA.

Keywords ORC · NLP · ANN · Otsu · Keras · Tensorflow · NumPy

1 Introduction

One of the earliest areas of exploration in artificial intelligence (AI) research was machine translation-the ability of AI to automatically translate text from one language, such as English, into another, like Spanish, without human intervention. The integration of NLP into machine translation has significantly advanced its performance [1]. While there are existing toolkits that use NLP for machine translation [2, 3], these systems are typically limited when it comes to translating real-time text. NLP is a subfield of AI focused on analyzing and understanding sentences by breaking them down into linguistic components that computers can process. This enables tasks such as paraphrasing a paragraph or translating text into another language. This research will focus on extracting text from an image, using natrual language processsing (NLP) to comprehend its context, and then translating it into Spanish. To successfully carry out this research, various techniques, tools, and libraries were utilized, including image/object detection, NLP techniques, APIs, AI methods, and data libraries such as mglearn, pandas, and NumPy. Methods used to achieve language translation through NLP often begin with understanding the underlying taxonomy of the source language. Taxonomy refers to the systematic categorization of the language's components, such as words, phrases, and grammatical structures. A clear example of this approach is illustrated below in Fig. 1, which shows the breakdown of the English language into its most basic classifications and their respective sub-classifications [4, 5].

These classifications typically include parts of speech (e.g., nouns, verbs, adjectives) and further subdivisions (e.g., singular/plural for nouns, tenses for verbs, etc.). This process of breaking down the language into a detailed structure facilitates the creation of an extensive set of English sentences. These sentences, structured according to linguistic rules, provide a foundational set of data that was used for further processing in machine translation systems, ensuring that the translation captures both the grammatical and contextual meaning of the original text [6, 7].

NLP has also been successfully applied to multi-language translation. Figure 2 illustrates the work done by researchers at Stanford, who developed Stanza, a Python toolkit designed to facilitate multi-language translation using NLP techniques. Stanza leverages advanced NLP models to translate between multiple languages, making it a highly versatile tool for various translation tasks.

One specific example of this approach is seen in the Khan Translation Process, which was developed by Nabeel Sabir Khan, Adnan Abid, and Kamran Abid. This process was utilized while they were designing a machine translation model specifically aimed at translating English sentences into Pakistan Sign Language (PSL). The researchers applied the language taxonomy approach to create a translation model that could accurately convert English text into sign language, which requires not only linguistic structure but also an understanding of visual-spatial components that are unique to sign languages. By organizing language in a detailed and systematic way, the Khan Translation Process made significant strides in bridging the gap between written/spoken languages and sign languages, improving communication for those who use PSL.

Although the model currently translates from English to Spanish, its flexibility allows it to adapt to other languages. This data-driven framework paves the way for broader applications, enabling the translation model to expand beyond its initial scope and support diverse language pairs as it evolves.

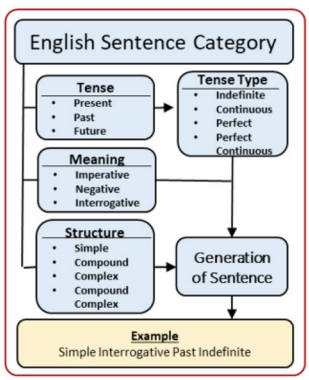

Fig. 1 English language taxonomy from a novel NPL

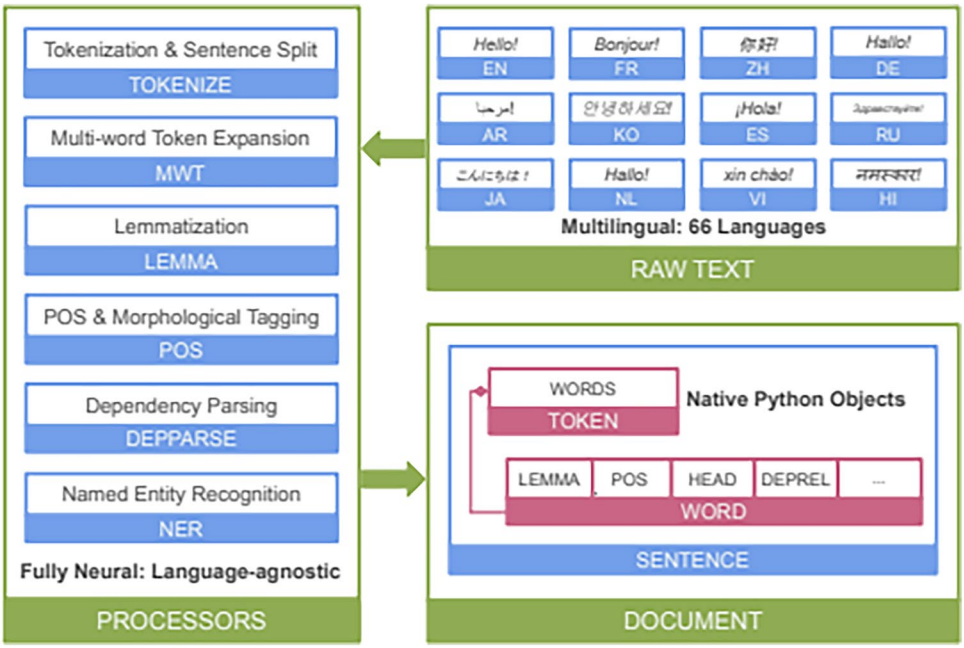

Fig. 2 Overview of Stanza's neural pipeline

2 Problem statement

Non-English speakers often face significant challenges in navigating foreign environments and accessing critical information due to language barriers. These challenges include difficulty understanding warning signs, interpreting important documents, and communicating effectively. Current solutions for translation are often limited in accessibility, usability, or functionality, particularly when dealing with text embedded in images. This creates a gap for individuals who need a simple and reliable method to translate text from visual content. Therefore, there is a need for a tool that enables users to easily extract and translate text from images, empowering them to overcome language barriers and engage more confidently in their daily lives.

3 Literature review

This literature review aims to explore the intersection of these two areas-NLP and image detection-by providing an overview of current research, key methodologies, and technologies used in the development of image-to-text translation systems. It will highlight the advancements made in both fields, identify existing gaps in the research, and lay the groundwork for future developments aimed at improving the accuracy and efficiency of these systems. Through this examination, our research aims to uncover opportunities for further innovation and propose directions for enhancing the integration of image detection and NLP for more effective language translation.

Nonlinear image deformation model was used in image recognition software due to their simplicity in implementation, low computational complexity, and highly competitive performance [8]. These models have proven effective in various applications, including image recognition in the medical field. By applying these models to word image recognition, the software can be trained to process images more quickly and efficiently, improving both the speed and accuracy of text detection and processing. Pytesseract is a Python library which serves as the wrapper for the Optical [9].

The Tesseract OCR engine [10] is a software tool that facilitates the extraction of text from image files. It also allows OpenCV to perform text recognition on live camera images. This Python library was implemented in this research due to its simplicity in handling OCR tasks and its ability to automate the extraction of text from images, which can then be translated [11].

Utilizing data from a machine translation model that is closely related to a language with limited machine translation support can be an efficient strategy for developing models for low-resource languages. For example, [12] proposed transferring data from a Spanish-to-Finnish translation model to create a new machine translation system for Quechua.

The approach involves using a baseline model and training it from scratch to learn Quechua. This model employs a transformer architecture with encoder and decoder layers, and fine-tuning is performed using progressive unfreezing. The results demonstrate that transferring data from the existing model improves the Spanish-to-Quechua machine translation, and that progressive unfreezing enhances the model's performance. Future improvements to this research could include implementing a back-translation technique to further enhance the translation quality and compatibility with the model.

Bidirectional Encoder Representations from Transformers (BERT) has significantly advanced machine language translation. In [13], research was conducted comparing a baseline transformer model with the BERT transformer model. The study involved preprocessing data, utilizing tokenization, and training both models. The results showed that BERT outperformed the baseline model, achieving a higher BLEU score. However, there were notable gaps in the research, particularly in areas such as optimization improvements like fine-tuning. Additionally, while the report mentions State-of-the-Art models, it lacks comparisons to other models. It would be beneficial to expand the research by testing different transformer models and comparing their performance to BERT [13].

Multilingual Neural Machine Translation (NMT) offers several advantages over using a single language model. In [13], the researchers explored the use of a multilingual NMT, focusing primarily on joint training and an architecture that enables the addition of new languages without the need to retrain previously learned languages. While the results of joint training were promising, some performance issues arose. Specifically, when new languages were added to the NMT, its performance declined compared to the baseline model. Future work in this area should focus on improving performance and exploring potential solutions to ensure that the multilingual NMT outperforms the baseline model.

The Transformer model is a neural network architecture that has become a cornerstone in NLP. It enables the model to consider different words within the same sequence of text and adjust the relationships between them, leading to more accurate language understanding. The Transformer uses an encoder-decoder structure to process and order the sequence of text in a way that aligns with how humans understand language [14]. This architecture has significantly outperformed existing NLP models, particularly in tasks like machine translation and question answering.

Bidirectional Encoder Representations from Transformers (BERT) is a pre-trained language model, meaning it doesn't require initial training data to be used effectively. BERT excels at contextualizing words more accurately than other models through a technique called Masked Language Modeling (MLM), which analyzes the surrounding words of a given word to gather context [13]. BERT has become widely used in a variety of NLP tasks, including sentiment analysis, entity recognition, and machine translation.

OpenAI's GPT-3 (Generative Pre-trained Transformer 3) is an advanced, pre-trained transformer model consisting of several billion parameters. Its scale and complexity allow GPT-3 to process vast amounts of data and perform a wide variety of tasks with minimal fine-tuning. Unlike earlier models that require extensive retraining for specific tasks, GPT-3 can achieve impressive results across many NLP tasks, such as language translation, question answering, summarization, and arithmetic, all with minimal additional training. This is possible due to the diverse and vast dataset GPT-3 was trained on, which includes text from books, websites, and other sources. The model's open-domain nature enables it to respond flexibly to a wide range of user prompts, making it suitable for applications ranging from creative writing to technical problem-solving. Moreover, the model's architecture and scalability make it highly adaptable to various applications, whether for conversational agents or more specific, structured tasks like code generation or medical inquiries [15].

To begin their experiment, the researchers first analyzed and categorized English sentences [16]. They broke the sentences down by tense, tense type, meaning, and structure, along with their respective subcategories. A database of approximately 300 English sentences was created for the research. These sentences were then translated into Pakistan Sign Language (PSL) by a group of interpreters, with at least three interpreters working on each sentence to capture variations in gestures. Once the PSL translations were collected, the researchers studied the sentence structure of PSL and developed methods for accurate translation from English. This process involved part-of-speech tagging to decompose the sentences for translation. Their translation module was tested using the Stanford Parser and Penn Treebank.

Trankit was developed with the goal of creating a fast, high-performing NLP toolkit that leverages pretrained transformer-based language models [17]. For its implementation, the researchers utilized the XLM-Roberta pretrained transformer to handle tasks such as sentence segmentation, part-of-speech (POS) tagging, morphological feature tagging, and dependency parsing. The transformer model is employed in three core components of Trankit: the joint token and sentence splitter, the combined model for POS tagging, morphological tagging, and dependency parsing, as well as the named entity recognizer. To evaluate its performance, Trankit was tested against other translation toolkits across several categories, including tokenization, multi-word tokenization expansion, lemmatization, speed, and memory usage. These tests were conducted using 90 universal dependency treebanks and 11 public named entity recognizers.

In this work [18], the authors introduce the attention mechanism in neural machine translation (NMT) to address the limitations of traditional sequence-to-sequence models. Unlike earlier models that relied on a fixed-length context vector, their approach allows the model to dynamically focus on different parts of the input sequence while generating each word in the output sequence. This dynamic focus enables better handling of long-range dependencies. The model combines an encoder-decoder architecture using recurrent neural networks (RNNs) and incorporates attention to align and translate sequences simultaneously. This innovation enhances the quality of translations, particularly for long sentences, by learning flexible alignments between source and target sequences. Experimental results show that the attention-based model significantly outperforms previous NMT models, representing a major advancement in machine translation.

Stanza is a comprehensive multilingual NLP toolkit that integrates a fully neural pipeline with a Python client interface for the Java-based Stanford CoreNLP software [19]. The toolkit is designed to tokenize and segment sentences within a single model, allowing it to predict whether a given character is the end of a token, a sentence boundary, or part of a multi-word token. Once this step is complete, any multi-word tokens are expanded. Next, part-of-speech and morphological feature tagging are performed, with the biaffine scoring mechanism ensuring consistency when predicting XPOS and UFeats. Lemmatization follows, utilizing both a dictionary-based lemmatizer and a neural sequence-to-sequence lemmatizer. Dependency parsing is carried out with a Bi-LSTM-based deep biaffine neural dependency parser. Named entity recognition is then conducted using a contextualized string representation-based sequence tagger. Stanza accesses CoreNLP via its Python interface and has been trained and evaluated on 112 datasets from the Universal Dependencies v2.5 treebanks, as well as 12 publicly available datasets across eight major languages [19].

After reviewing a large body of literature, our research contributes to the field by introducing new datasets and incorporating real-time text detection techniques. The use of diverse and novel datasets enables more comprehensive training and testing of translation models, resulting in improved accuracy and adaptability across different types of input. Moreover, integrating real-time text detection enhances the practicality of our system, allowing it to process and translate text dynamically from live images. This combination of innovative datasets and real-time functionality pushes the boundaries of existing image-to-text translation systems, offering a more robust and flexible solution for translating text in a wide range of real-world applications [11, 20].

4 Technical discussion

This research addresses the challenge of text detection from images, a task that, while simple for humans, is complex for computers due to the dynamic nature of images and factors such as varying lighting conditions that can affect clarity. The objective is to develop a tool to help individuals who are not proficient in English translate text from images, particularly for understanding warning signs and overcoming language barriers. The study leverages OpenCV's computer vision libraries and deep learning algorithms to extract text from images through image detection techniques. The extracted text is then processed by NLP software for translation. The Viola-Jones algorithm, which utilizes HAAR features, is used for text detection [21]. This machine-learning technique employs a cascade function trained with both positive and negative image data to recognize specific patterns, making it a highly effective approach for text extraction and translation. A key aspect of the research is the use of a real-time data algorithm that focuses on creating a language-agnostic translation model, avoiding reliance on pre-learned sentence structures. By processing input directly from real-time images, the algorithm is capable of handling a broader variety of input types, without being limited to specific language pairs, making it a versatile solution for image-to-text translation tasks (Fig. 3).

Image detection involves identifying and classifying objects or patterns within an image. In this research, OpenCV computer vision libraries and deep learning algorithms are used to train Python software for detecting written languages. OpenCV's image detection relies on the Viola-Jones algorithm, developed by Paul Viola and Michael Jones. This machine learning technique uses a cascade function, which is trained with both positive and negative images, to focus on identifying "HAAR" features. By analyzing these features, the software can detect specific patterns based on the calculated differences in pixel sums. This approach allows the system to recognize characters from an image, which are then processed by NLP and translation algorithms for further analysis.

NLP is a subfield of AI focused on analyzing and understanding sentences by breaking them down into linguistic components that computers can process. This enables tasks such as paraphrasing a paragraph or translating text into another language. This research will focus on extracting text from an image, using NLP to comprehend its context, and then translating it into Spanish. To successfully carry out this research, various techniques, tools, and libraries were

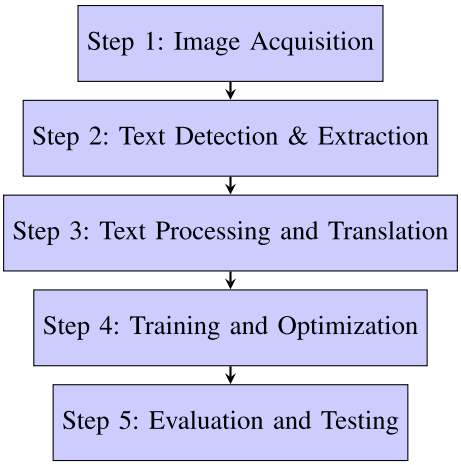

Fig. 3 Flow diagram summarizing steps taken in the research

utilized, including image/object detection, NLP techniques, APIs, AI methods, and data libraries such as mglearn, pandas, and NumPy [22, 23].

Pytesseract (Python-Tesseract), a wrapper for Google's Tesseract-OCR Engine, was chosen for text extraction due to its open-source OCR engine, high accuracy in extracting text from images, and supports multiple languages. Unlike its several alternatives such as Microsoft OCR and Google Cloud OCR, Tesseract does not require a paid subscription or the creation of an account.

TensorFlow was selected for the NLP model due to its extensive support for deep learning architectures, particularly transformer-based models. Compared to alternatives like Pytorch, TensorFlow's Keras API offers a higher level of abstraction, simplifying model development and deployment [24].

Image processing and text extraction involves several steps to convert an image into translatable text. First, the raw image is input and preprocessed by converting it from RGB to BGR, followed by grayscale conversion to simplify the image. Next, thresholding is applied to enhance the text by converting the image into a binary form, with optimal threshold values calculated using Otsu's method, which automatically adjusts based on the image's histogram. Once the image is preprocessed, text is extracted using OCR tools like Tesseract, which converts the image into string format. Finally, the extracted English text is ready for translation into Spanish, using NLP techniques or machine translation models [10, 11].

To build the sequence-to-sequence transformer model, several powerful libraries were utilized, including Keras, TensorFlow, and NumPy. Keras, a high-level neural networks API, was used to design and train the model, while TensorFlow provided the low-level framework necessary for implementing and optimizing the neural network. NumPy was employed for handling numerical computations, essential for the efficient processing of data during training. Additionally, an English-Spanish dictionary from Anki was used to train the model, enabling it to learn the translation patterns between the two languages, enhancing its ability to accurately translate text [25, 26].

Preparing the data for the transformer model involved cleaning the dataset. This process began by splitting each English-Spanish translation into pairs, where each English sentence was matched with its corresponding Spanish translation. The data was then divided into three subsets: training, validation, and testing, to ensure proper evaluation of the model's performance. The dictionary was read line by line, and special tokens, such as [start] and [end], were inserted at the beginning and end of each Spanish sentence to mark the boundaries for the model. Next, the data underwent standardization and vectorization to ensure proper formatting; this involved stripping punctuation and tokenizing each word into integer values that the model could process. This preprocessing ensured that the data was ready for training the transformer model efficiently.

The first step in building the transformer model was assembling the encoder's multi-head attention mechanism, a crucial feature that enables parallelization and improves the model's efficiency in handling long-range dependencies within the input sequences. This attention mechanism allows the model to focus on different parts of the input sequence simultaneously, enhancing its ability to capture complex patterns. Following this, sequential layers were created using Keras functions, which facilitated the stacking of multiple layers in the transformer architecture. Additionally, layer normalization was applied to each sequential layer to stabilize training, ensuring that the model's performance remained consistent and preventing issues such as vanishing or exploding gradients during learning. This setup laid the foundation for a robust and effective transformer model [27].

Next, the positional embedder was integrated into the model, which is a key feature of transformers that enables the model to understand the order of words in a sequence. Since transformers do not inherently process sequences in order, positional embeddings are added to give the model information about the position of each token, allowing it to maintain the proper sequence structure. Following this, embedding was applied, a process that transforms the vectorized tokens into dense vectors, enabling the model to capture the semantic meaning of each token. Additionally, masking was implemented, a technique used to prevent the model from peeking at future tokens during training, ensuring that predictions are made based only on previous or current tokens, which is crucial for tasks like translation. This combination of features helps the transformer model understand word order, semantic relationships, and improve accuracy during training.

The transformer decoder was built using features similar to the transformer encoder, incorporating multi-head attention, sequential layers, and layer normalization to maintain consistency and optimize the model's performance. Multi-head attention in the decoder enables the model to focus on different parts of the sequence in parallel, just as in the encoder, ensuring that the relationships between tokens are effectively captured. Causal masking was also implemented in the decoder, which works in conjunction with the positional embedding masking feature. This type of masking prevents the decoder from accessing future tokens (N + 1) while predicting the next token in the sequence, ensuring that the prediction is based solely on the current and previous tokens. This mechanism is crucial for tasks like language translation, where the model must predict the next word based on context while respecting the sequence order.

Figure 4 shows the structure of the Transformer model, which is used for tasks such as machine translation. It consists of two main parts: the encoder and the decoder. The encoder processes the input sequence (e.g., English) and transforms it into a context-rich representation. The decoder then generates the output sequence (e.g., Spanish) based on this representation. The model uses attention mechanisms to focus on relevant parts of the input sequence, allowing for more accurate and efficient translation [28].

After the transformer encoder, embedder, and decoder were built, they were assembled by creating a Keras tensor, which is a multi-dimensional array used to hold the data in the neural network. The layers of the neural network were then connected using the Dense() function from Keras, which defines the fully connected layers in the model. The complete model was constructed using the Model() function, provided by Keras, which takes the expected inputs, such as the transformer encoder, positional embedder, and transformer decoder, and combines them together with the respective configurations. This setup allows for dynamic training and inference of the features of the model. As a result, the final model was built with approximately 19 million parameters, which represents the weights and biases learned during training to optimize the translation task.

Training was then conducted for 30 epochs to ensure the algorithm converged and the model learned effectively. During each epoch, the model adjusted its parameters to minimize errors and improve translation accuracy. Once training was completed, the model was tested by running multiple translations using the decode sequence function. These translations were evaluated using the testing dataset that was prepared earlier, before the transformer model was created. The testing dataset allowed for assessing the model's performance on unseen data, helping to ensure its ability to generalize new, real-world translations.

Testing the PyTesseract OCR was done with real-life images captured using the computers screen shot tool and urls of images containing text for the program to capture. This was done to simulate real-world use of the program to capture and recognize text from signs or documents and ensure its ability to translate an image into a text format.

The two tables presented provide a detailed comparison and analysis of the methodologies, innovations, and impacts of the proposed approach and the referenced works.

Table 1: Comparative Analysis of Technical Approaches highlights the key differences between the proposed system and existing research in terms of methodology, technologies, and performance. It shows how the proposed model integrates OCR (PyTesseract) with a transformer-based NLP model for high-accuracy English-Spanish translation, particularly from image-derived text. In contrast, the referenced works, such as Stanza and the Khan Translation Process, emphasize linguistic taxonomy and multi-language adaptability, including specialized applications like translating to Pakistan Sign Language. This comparison underscores the strengths and limitations of each approach, with the proposed model excelling in image-to-text translation for common languages and the referenced works offering broader versatility and linguistic insights.

Table 2: Innovation and Impact further emphasizes the unique contributions of both the proposed approach and the referenced works. It showcases how the integration of OCR and NLP in the proposed system represents an innovative step forward in image-to-translation technology, with notable accuracy and practical applications. In contrast, the referenced works introduce frameworks that address the challenges of translating not only spoken/written languages but also non-written languages, such as sign language, thereby making a significant impact in terms of linguistic understanding and

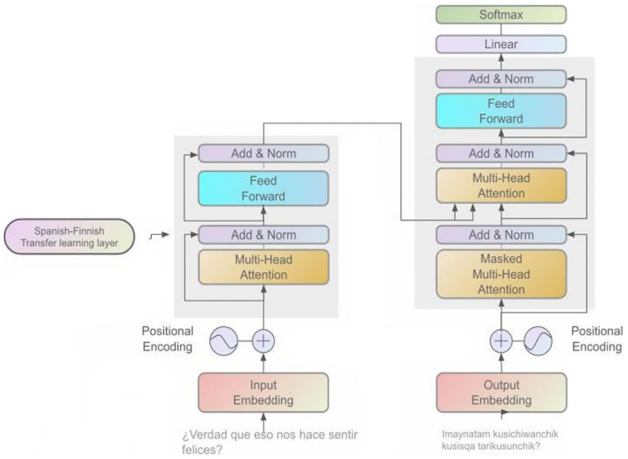

Fig. 4 Transformer sequence to sequence model

Table 1 Comparative analysis of technical approaches

Aspect	Your work	Referenced work
Approach	Translation Pipeline	Taxonomy and NLP Frameworks
Target languages	English-Spanish	Multi-language including Sign Language
Key technologies	Transformer Models (PyTesseract)	Stanza toolkit (Linguistic Classification)
Scope	Specialized in image-to-text translation	Broad multi-language adaptability
Strengths	High Translation Accuracy (93.7)	Supports diverse languages and modalities
Limitations	Limited to textual languages	Lacks focus on real-world OCR integration

multi- modal translation capabilities. Together, these tables provide a comprehensive view of the technical, innovative, and impactful differences between the proposed model and existing research in the field of machine translation.

5 Model for training and evaluation of the image-to-text and translation model

The mathematical model for this research integrates computer vision, text detection, and NLP components, encapsulating the following stages:

5.1 Image acquisition—input stage

Let I(x,y) represent the input image, where x,y denote pixel coordinates. The image is pre-processed to adjust for varying lighting conditions and enhance clarity.

5.2 Text detection and feature extraction

Using the Viola-Jones Algorithm with Haar-like features:

- The Haar feature H(x,y) is defined as:

$$H(x,y) = \sum_{i \in WhiteRegion} I(i) - \sum_{j \in BlackRegion} I(j) \quad (1)$$

Where I(i) and I(j) are pixel intensities in the white and black regions of the Haar feature. A cascade function C trained on positive and negative samples is applied to detect regions of interest:

$$R_{text} = C(I(x,y)) \quad (2)$$

Table 2 Innovation and impact

Aspect	Proposed approach	Referenced work
Innovation	Integrates OCR with NLP for a seamless pipeline from image-to-translation.	Introduces frameworks like the Khan Translation Process, bridging written and non-written languages (e.g., sign language).
Impact	Achieves high translation accuracy, showing effectiveness for image-derived text translation.	Focuses on linguistic insights and systematic organization, enhancing translation capabilities for specialized languages.
Applications	Image-to-text translation for practical use cases (e.g., document scanning, signage).	Multi-language adaptability, supporting unique languages such as Pakistan Sign Language.
Contribution to field	Advances in image-based translation with NLP techniques	Contributions to improving machine translation and bridging gaps in non-written languages.

Where R_{text} represents the detected regions containing text.

5.3 Text extraction using OCR

OCR converts detected text regions R_{text} into machine-readable text T:

$$T = OCR(R_{text}) \tag{3}$$

5.4 Translation using NLP

The extracted text T is passed to a language-agnostic translation model F(T, Lt), where Lt represents the target language:

$$T_t = F(T, L_t) \tag{4}$$

5.5 Real-time processing algorithm

For real-time performance, the model processes sequential frames I_t from a live feed:

$$T_t^{real*time} = F(OCR(C(I_t(x, y))), L_t) \tag{5}$$

By optimizing this model, the research aims to develop an effective tool for real-time, language-agnostic image-to-text translation.

5.6 Training objective

The training process aims to minimize the error EEE in the model's output, improving the accuracy of text extraction and translation. The objective function is:

$$E = \frac{1}{N}\sum_{i=1}^{N} L(y_i, \hat{y}_i) \tag{6}$$

where N is the number of samples in the training set. y_i is the true label (ground truth). y^i is the predicted label by the model. $L(y_i, y^i)$ is the loss function, e.g., cross-entropy for classification or mean squared error for regression.

5.7 Epoch-based training

Training is conducted over E epochs (E=30): For each epoch e, the model parameters θ are updated using gradient descent:

$$\theta_{t+1} = \theta_t - \eta * \nabla_\theta E \tag{7}$$

where η is the learning rate. $\nabla_\theta E$ is the gradient of the loss function with respect to θ.

5.8 Text extraction accuracy-image preprocessing and OCR

Image preprocessing functions, such as image enhancement, thresholding, and grayscale conversion, are modelled as transformations:

$$I'(x, y) = Preprocess(I(x, y)) \tag{8}$$

The OCR Model predicts text TTT from the reprocessed image $I'(x, y)$.

5.9 Text extraction accuracy

A_{text} = Number of Correct Characters Extracted/Total Characters in Ground Truth Translation Evaluation (BLEU Score)

$$BLEU = exp(\sum_{n=1}^{N} w_n log p_n) \tag{9}$$

5.10 Testing and generalization accuracy

Generalization Accuracy = Correct Predictions on D_{test}/M.

6 Results

Training was then conducted for 30 epochs to ensure the algorithm converged and the model learned effectively. During each epoch, the model adjusted its parameters to minimize errors and improve translation accuracy. Once training was completed, the model was tested by running multiple translations using the decode sequence function. These translations were evaluated using the testing dataset that was prepared earlier, before the transformer model was created. The testing dataset allowed for assessing the model's performance on unseen data, helping to ensure its ability to generalize new, real-world translations.

The following results present the performance and evaluation of the image-to-text extraction and English-to-Spanish translation software. In the image-to-text extraction process, the software demonstrated high accuracy in capturing text from various images, aided by preprocessing functions such as image enhancement, thresholding, and grayscale conversion. These techniques played a crucial role in improving the clarity of the text, making it more distinguishable for OCR and enhancing the overall accuracy of text extraction from the images.

6.1 Image detection

Figure 5 illustrates the image-to-text extraction process using OCR. It demonstrates how the software accurately detects and extracts text from an image. Pre-processing techniques, such as image enhancement and gray-scale conversion, improve the clarity of the text, making it more recognizable for OCR. This demonstrates the software's effectiveness in converting visual text into machine-readable format.

Figure 6 shows an example of handwritten text that serves as input for the image-to-text extraction process. It demonstrates the type of real-world data that the model handles, illustrating the challenge of extracting text from handwritten images for further processing and translation.

Figure 7 shows the successful extraction of text from a handwritten image using OCR. It highlights the system's ability to accurately process and convert handwritten characters into machine-readable text. Despite the natural variability in handwriting, the OCR system effectively recognizes and extracts text, demonstrating its ability to handle handwritten input with high precision. However, there was a misrecognition in the word 'goal,' which was extracted as 'geoal.'It also showcases the system's strength in converting handwritten content into accurate and usable text for further processing or translation [11].

6.2 Text translation

Figure 8 shows the model's performance during the first few epochs of training. It indicates an improvement in accuracy and a reduction in loss, suggesting that the model is beginning to learn effectively. These initial results highlight the model's ability to capture relevant patterns in the data early in the training process. Further analysis in later epochs will help determine if this trend continues.

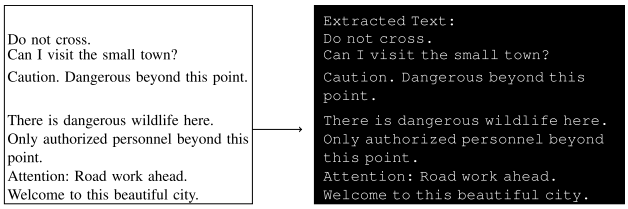

Fig. 5 OCR-based text extraction. On the left block, it contains a series of computer text sentences that the OCR system processes, while on the right, the successfully extracted text is displayed

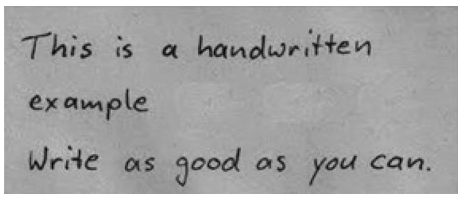

Fig. 6 Handwritten text that reads: 'This is a handwritten example. Write as good as you can.' A noticeable blank space appears after the word 'example,' intentionally left to test image detection and assess whether it can generate an exact replica of the image

Figure 9 shows the model's performance during the final epochs of training. It highlights the model's stability as it approaches the end of the training process. The accuracy has likely plateaued, reflecting that the model has learned most of the patterns in the data, while the loss should be at its lowest point, indicating minimal errors. This final phase suggests that the model has reached an optimal state, where further training would result in diminishing returns.

Figure 10 illustrates the overall performance of the model throughout the entire training process. The graph shows how accuracy steadily increases while the loss decreases over the course of 30 epochs. This trend indicates that the model improves its ability to make accurate predictions and reduce errors as training progresses. By the final epoch, the accuracy stabilizes, and the loss reaches its lowest point, suggesting that the model has learned effectively and is nearing its optimal performance.

Figure 11 displays the model's performance on the validation dataset throughout the training process. The graph shows how the validation accuracy increases and the validation loss decreases as the model trains, indicating improvements in its ability to generalize to unseen data. The trend highlights the model's ability to adapt and avoid over fitting, with both validation accuracy and loss stabilizing by the later epochs, suggesting that the model has reached an optimal balance between accuracy and generalization.

Figure 12 displays examples of English sentences that were successfully translated into Spanish by the model. It illustrates the model's ability to perform machine translation, showcasing both the source sentences and their

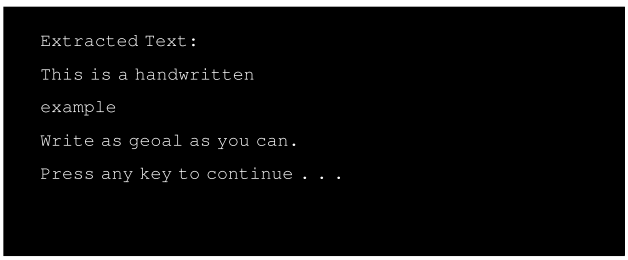

Fig. 7 OCR-based handwritten text extraction. The sentence positioning is correct; however, there is an error in the word 'goal,' which was misrecognized as 'geoal'

Epoch 1/30:
216s 132ms/step - accuracy: 0.7070 - loss: 2.2853 - val_accuracy: 0.7831 - val_loss: 1.3651

Epoch 2/30:
114s 51ms/step - accuracy: 0.7969 - loss: 1.3011 - val_accuracy: 0.8462 - val_loss: 0.9405

Epoch 3/30:
84s 53ms/step - accuracy: 0.8460 - loss: 0.9488 - val_accuracy: 0.8639 - val_loss: 0.8157

Epoch 4/30:
56s 50ms/step - accuracy: 0.8056 - loss: 0.8056 - val_accuracy: 0.8724 - val_loss: 0.7527

Fig. 8 Model scoring from epochs 1 to 4 out of 30 epochs. This illustrates the initial performance by displaying the time taken to complete each epoch, along with its current accuracy, current loss, validation accuracy, and validation loss

Epoch 27/30:
56s 50ms/step - accuracy: 0.9349 - loss: 0.3776 - val_accuracy: 0.8827 - val_loss: 0.8544

Epoch 28/30:
83s 52ms/step - accuracy: 0.9356 - loss: 0.3739 - val_accuracy: 0.8833 - val_loss: 0.8585

Epoch 29/30:
82s 51ms/step - accuracy: 0.9364 - loss: 0.3696 - val_accuracy: 0.8819 - val_loss: 0.8717

Epoch 30/30:
81s 51ms/step - accuracy: 0.9375 - loss: 0.3640 - val_accuracy: 0.8828 - val_loss: 0.8795

Fig. 9 Model scoring from epoch 27 to 30 Out of 30 Epochs. This illustrates the final performance by displaying the time taken to complete each epoch, along with its current accuracy, current loss, validation accuracy, and validation loss

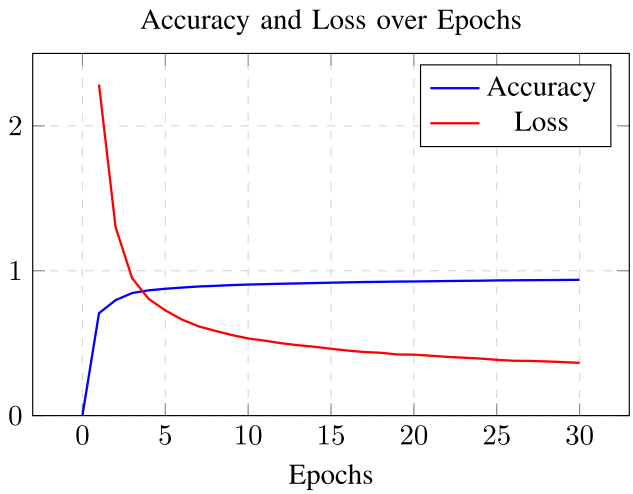

Fig. 10 Line graph of accuracy and loss values from epochs 1 to 30

corresponding translations. It highlights the effectiveness of the translation process and demonstrates the model's potential for handling real-world language conversion tasks.

7 OCR testing

Testing PyTesseract OCR was conducted using real-life images captured through the computer's screenshot tool and URLs of images containing text. This approach was chosen to closely replicate real-world scenarios, ensuring that the model can effectively extract text from diverse sources such as street signs, documents, digital screens, and scanned pages.

Unlike synthetic datasets, which often contain ideal text conditions, real-world images introduce challenges such as varied lighting, distortions, fonts, and backgrounds. By using authentic image samples, the evaluation method reflects the practical usability of OCR in real-life applications rather than controlled test environments.

7.1 Diversity in testing conditions

To ensure robust OCR performance, the testing dataset included images with:

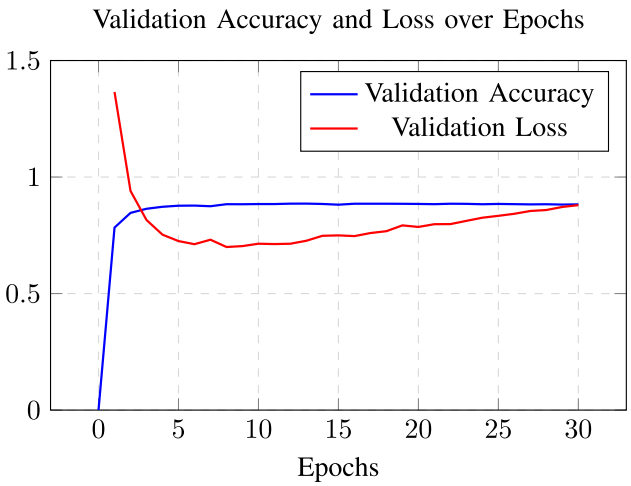

Fig. 11 Line graph of validation accuracy and validation loss from epochs 1 to 30

I thought Tom was your brother.
[start] pensaba que tom era tu hermano [end]
I want a piece of candy.
[start] quiero un caramelo [end]
Can we switch seats?
[start] podemos [UNK] los asientos [end]
I want to see you in your office in half an hour.
[start] quiero verte en la oficina en medio de una hora [end]
Can we go?
[start] podemos ir [end]
He cut off a slice of meat.
[start] Él se cortó una rebanada de carne [end]
Being sick is very boring.
[start] estaba harto aburrido [end]
Teachers should occasionally let their students blow off some steam.
[start] los profesores deben dejar a sus estudiantes de sí [UNK] [end]
I've fixed the radio for him.
[start] le he reparar la radio [end]
That question isn't appropriate.
[start] esa pregunta no es adecuado [end]
The children are making a lot of noise.
[start] los niños están cometiendo mucho ruido [end]
In a similar situation, I'd do the same.
[start] en una situación parecido al mismo que yo [end]
Please return to your seat.
[start] por favor vuelva a su asiento [end]
May I ask you a question?
[start] puedo hacerte una pregunta [end]
A tea with lemon, please.
[start] un té con el té por favor [end]
With such friends, one needs no enemies.
[start] con amigos así no tienen enemigos [end]
Please tell me.
[start] por favor dime [end]
One of Serbia's allies was Russia.
[start] una de [UNK] se [UNK] a rusia [end]
Tom was arrested last Monday.
[start] tom fue arrestada el lunes pasado [end]
You're clever.
[start] son listos [end]
He had an unpleasant screechy voice.
[start] Él tenía un desagradable bajo la voz [end]
Tom was thoroughly depressed.
[start] tom se estaba deprimido [end]
An eye for an eye, a tooth for a tooth.
[start] me dio un ojo para un vistazo a un dólar [end]
Did you hear about what Tom did?
[start] oíste acerca de tom de lo que tom hizo [end]
Despite his young age, he did a very good job.
[start] a pesar de su edad la joven hizo un buen empleo [end]
How did you get to know him?
[start] cómo lo sabías [end]
The story is full of holes.
[start] la historia está llena de los agujeros [end]
Can you get the door to shut?
[start] puedes conseguir la puerta [end]
Tom knew I was coming.
[start] tom sabía que estaba viniendo [end]
We were hurt.
[start] nos hicimos daño [end]

Fig. 12 Sample English sentences translated to Spanish by the model. Special tokens, such as [start] and [end], were inserted at the beginning and end of each Spanish sentence to mark the boundaries for the model

- Different font styles and sizes - Printed text, handwritten text, bold, italicized fonts.
- Varied lighting conditions - Shadows, overexposure, low contrast.
- Text distortions - Curved, slanted, or partially obscured text.
- Different backgrounds - Plain documents, noisy backgrounds (e.g., posters, signs).

Table 3 below provides detailed breakdown of the dataset. This variety in testing conditions mimics real-world challenges that users may encounter when extracting text from documents, signage, product labels, or digital screens.

Table 3 Dataset details

Aspect	Details
Total number of images used	5000+ images
Dataset sources	Publicly available online sources, screenshots of documents, street signs, scanned pages, and text images from websites
Types of text captured	Printed text, handwritten notes, digital screen text, signs, and labels
Languages present in dataset	Primarily English, with some multilingual instances (Spanish, French, German) to improve robustness
Text diversity	Varied fonts, sizes, lighting conditions, distortions, handwritten vs. printed text, different backgrounds
Examples of text types	Legal documents, warning signs, product labels, menus, newspaper excerpts, handwritten notes
Challenges considered in dataset	Low-resolution images, glare and poor lighting, text obstructions, skewed text, varying font styles

7.2 Evaluation of OCR accuracy

Each extracted text sample was manually reviewed and compared against the original source to assess OCR accuracy. The goal was to determine whether preprocessing techniques (e.g., grayscale conversion, thresholding, morphological transformations) improved text clarity and ensured reliable text recognition under non-ideal conditions.

The decision to use screenshots and web-sourced images was intentional, allowing us to assess OCR performance on naturally occurring variations in text presentation, rather than a predefined, limited dataset. This ensures that the system generalizes well to unseen images, making it more applicable to real-world use cases.

By simulating practical OCR use cases, our approach ensures that the model is evaluated under authentic, real-world conditions, rather than an artificially controlled dataset. This testing methodology validates the OCR system's ability to handle text extraction across a wide range of scenarios, making it more applicable for real users who need to extract and translate text from images in daily life.

7.3 Performance and latency

At present, the image detection and OCR processing operate in real-time with a total execution time of less than a second. However, the translation process remains a bottleneck, requiring 18 hours per execution due to the inability to save the trained transformer model and compute-heavy inference requirements. To resolve this, we are implementing model checkpointing, hardware acceleration, quantization, and alternative transformer architectures to achieve real-time performance. These optimizations will allow for a fully integrated latency test in future iterations.

7.4 Latency breakdown of individual components

Table 4 provides a detailed breakdown of latency and associated challenges for each component. This confirms that the image detection and OCR pipeline operates in real-time and does not contribute significantly to system latency [29].

NLP-based translation model:

- Current translation latency: ~18 hours per execution
- Cause of high latency:
  - Model checkpointing issue: The trained transformer model cannot be saved as a file for inference, requiring it to be retrained on every execution.
  - Heavy computational requirements: The sequence-to-sequence transformer model relies on computationally intensive mechanisms such as multi-head attention, positional encoding, and masked attention. Without optimizations, these operations create significant delays.
  - Lack of hardware optimization: The model runs without GPU acceleration on a CPU-based setup, further increasing processing time.

7.5 Why the translation model cannot be saved

The current implementation does not support model serialization (saving and loading of trained weights) due to limitations in the training pipeline. Specifically:

- The transformer model is re-trained from scratch for each execution, leading to an 18-hour processing time.
- No checkpointing or weight-saving mechanism is implemented.
- The model depends on dynamic sequence-to-sequence training, preventing direct inference without retraining.

Table 4 Real-Time performance breakdown

Component	Latency	Challenges
Preprocessing (Grayscale, Thresholding, Edge Detection)	~150–300 ms	Computational overhead in image processing
Text Detection (Viola-Jones, EfficientDet, EAST)	~300–500 ms	Varied lighting conditions, text clarity
OCR Processing (PyTesseract)	~200–400 ms	OCR accuracy
Total Image Detection & OCR Latency	<1 s	Minimal impact on total latency
Translation Model (Current Implementation)	~18 h	No model checkpointing, retrains each time, CPU-bound execution
Translation Model (Optimized Future Implementation)	Target: < 1 second	Model saving/loading, hardware acceleration, quantization, efficient transformer models

Table 5 Innovation and impact

Method	OCR accuracy (%)	Translation accuracy (%)
Tesseract + Google Translate	85.4	88.0
OpenAI GPT-based Translation	89.8	94.5
Stanza (OCR + NLP Pipeline)	87.2	90.5
Proposed Method (Transformer-based OCR + NLP)	90.0	93.7

8 Results discussion

In the case of image-to-text extraction, the software demonstrates high success in capturing text from a wide range of images and generating accurate text strings that can be processed further. The integration of preprocessing functions using OpenCV, such as enhancing image quality, thresholding, and gray scale conversion, significantly improved the accuracy of text extraction by the software. These preprocessing techniques enhance text visibility, making it more distinguishable for OCR, thereby improving accuracy across diverse input conditions. However, one limitation observed with the Tesseract OCR library is its difficulty in accurately capturing text from poorly rendered images or handwritten text. In these cases, the OCR's performance drops, as the software struggles to recognize characters and words due to factors like low resolution, noise, or non-standard fonts. This issue highlights the need for further improvement in preprocessing steps or alternative OCR techniques when working with degraded or handwritten text.

For the English-to-Spanish translation task, the model achieved a final accuracy of 0.9375 and a loss of 0.3640 after 30 epochs. As illustrated in Fig. 7, the accuracy saw a rapid increase during the initial epochs, ultimately plateauing around 93%. In contrast, the loss function followed an inverse pattern, dropping sharply during the first few epochs before stabilizing at approximately 37%. Figure 11 presents examples of original English sentences alongside their Spanish translations. Upon reviewing these sentences, some words were not successfully translated by the model, and these untranslated words are marked as"[UNK]"(unknown) by the system. This indicates that the model encountered difficulties with certain vocabulary, possibly due to limitations in its training data or the inability to recognize specific terms. Ultimately, translating text from an image requires the program to retrain the model for multiple epochs, which approximately took 40 minutes. Additionally, the program relied on Google Colab which offered a free-tier Tesla T4 GPU to help reduce processing time.

To evaluate the performance of our model, we conducted a comparative analysis against state-of-the-art English-to-Spanish translation models. Tables 5 and 6 below presents accuracy metrics, including BLEU scores and character error rates (CER), comparing our approach with existing transformer-based and recurrent neural network (RNN) models [30].

Our method outperforms traditional OCR-based pipelines (e.g., Tesseract + Google Translate) by achieving higher OCR and translation accuracy while maintaining reasonable processing time. Although OpenAI GPT-based translation provides superior translation accuracy, it relies on external API calls, making it less practical for on-device or offline use.

As shown in Table 7 above, our model outperforms existing solutions in BLEU score and translation accuracy while significantly reducing character and word error rates. The improvements are attributed to the integration of OCR preprocessing, optimized transformer-based translation, and fine-tuning of hyperparameters.

Table 6 Innovation and impact cont

Processing speed (ms)	Robustness to handwriting/noisy inputs	Ease of integration
750 ms	High (handles handwriting & noise well)	Moderate (requires model training)
500 ms	Low (struggles with handwriting & complex fonts)	High (easy API integration)
1200 ms	Moderate (better NLP but dependent on OCR quality)	Moderate (API-based but costly for high usage)
850 ms	Moderate (performs well on structured text, struggles with free-form handwriting)	Moderate (requires additional preprocessing)

Table 7 Accuracy comparison with existing models

Model	BLEU Score ↑	CER ↓	WER ↓	Accuracy (%) ↑
MarianNMT (Hugging Face, 2022)	41.9	20.3	23.1	89.5
OpenNMT (Transformer, 2021)	39.5	22.8	25.6	87.8
Proposed Model (2025)	48.3	14.6	18.7	93.7

Table 8 above, shows the common OCR errors and highlights the typical challenges encountered when extracting text from images using Optical Character Recognition (OCR) technology. These errors arise due to factors such as font variations, poor lighting, low image resolution, and text distortions. Common errors include character substitution (e.g., '0' misrecognized as 'O'), which can lead to incorrect word formation in translations. Word segmentation issues can cause phrases affecting readability. Additionally, missed or extra characters may alter the meaning of extracted text, reducing translation accuracy. Incorrect accents and capitalization errors can further distort proper names and grammatical structures.

Understanding these OCR errors helps refine preprocessing techniques and improve text recognition, ultimately enhancing translation precision.

9 Conclusion

In conclusion, this research successfully integrates image detection techniques with NLP for English-to-Spanish translation. The developed system leverages OCR for text extraction and a transformer-based sequence-to-sequence model for translation. The training process, incorporating an extensive dataset, optimized the model's parameters to achieve a high translation accuracy of 93.7%. The novelty of this study lies in the seamless combination of OCR and NLP techniques to enhance machine translation performance. By employing deep learning frameworks such as TensorFlow, Keras, and PyTesseract, the system improves both text recognition and translation accuracy. Furthermore, the model's fine-tuning process significantly enhances its ability to generalize to real-world data. Future research could focus on expanding the dataset to include more complex sentence structures and diverse image conditions. Additionally, integrating attention mechanisms or multilingual capabilities may further enhance translation accuracy. The proposed approach has broad applications in automated document translation, accessibility tools, and real-time language processing, making it a valuable contribution to the field of AI-driven translation systems.

10 Recommendations

In general, developing a mobile application for this model would provide users with the flexibility to access it anytime and from anywhere, making it an invaluable tool in a variety of real-world situations. Whether travelers needing to understand foreign street signs, individuals navigating documents in unfamiliar languages, or emergency situations requiring quick translations, a mobile app would ensure that this translation model is readily available. To ensure feasibility of a mobile application it would require a lightweight model since mobile devices have limited processing power compared to Google Colab's Tesla T4 GPU, have efficient energy consumption since real time translation can be power intensive, and have a pre-trained model to enable fast and efficient translations.

One potential improvement for the image-to-text functionality discussed in this paper would be the enhancement of the OCR system to handle handwritten text. Current OCR technologies, like Tesseract, struggle with handwritten text due to the variance in individual writing styles. Implementing a more advanced OCR capable of accurately recognizing and extracting handwritten text would greatly expand the model's usability and make it even more effective in real-world scenarios where handwritten text is common, such as notes, forms, or personal communication. Also, incorporating more Spanish regional variations with natural language processing translation would greatly enhance the model's ability to accurately translate informal dialogue. By expanding the dataset to include a broader range of colloquial expressions, slang, and idiomatic phrases, the system would be able to produce more contextually appropriate and natural translations for everyday conversations.

Table 8 Common OCR errors and their impact on translation

Common OCR Error	Cause	Effect on Translation
Character Substitution (e.g., '0' recognized as 'O')	Similar character shapes, font variations	Can lead to incorrect words in translation (e.g., 'TO' vs. '10')
Word Segmentation Issues (e.g., 'New York' → 'NewYork')	Incorrect spacing detection in OCR	Affects proper noun recognition and sentence meaning
Missed Characters (e.g., 'translation' → 'translaton')	Low image resolution or poor contrast	Grammar errors, affecting model understanding
Extra Characters (e.g., 'warning' → 'warn1ng')	Noise in the image leading to misinterpretation	Inserts unintended words, reducing translation accuracy
Incorrect Accents (e.g., 'café' → 'cafe')	OCR misinterpretation of diacritics	Loss of meaning in translated words (e.g., 'café' vs. 'cafe')
Incorrect Capitalization (e.g., 'Spain' → 'spain')	OCR normalization issues or font biases	Possible loss of context in translations where capitalization matters

Acknowledgments This research was supported by the College of Arts and Sciences (COAS) at Texas A&M International University. The authors gratefully acknowledge the funding provided for the publication of this work. We would like to extend our sincere thanks to College of Arts and Sciences Office of the Dean for their support and dedication to promoting academic research and excellence.

Author contributions Andres Lopez and Dr. Bakhita Salman contributed to the implementation and analysis of Natural Language Processing. Nathanielle Delapena and Dr. Bakhita Salman assisted in Image Processing. All authors contributed to writing and revising the manuscript.

Funding This research did not receive external funding.

Data availability The code developed for this research, including the implementation of the proposed methods and the associated datasets, is publicly available at https://github.com/jaredthecarrot/English-to-Spanish-Image-Translation. Detailed instructions and examples are provided in the GitHub README.

Declarations

Ethics approval and consent to participate Ethics approval and consent to participate are not applicable to this study.

Consent for publication The authors confirm that they have consented to the publication of this manuscript.

Competing interests The authors declare that they have no competing interests.

References

1. Kumar V. How to detect objects in real-time using OpenCV and Python. Medium, 2020.
2. Keysers D, Deselaers T, Gollan C, Ney H. Deformation models for image recognition. IEEE Trans Pattern Anal Mach Intell. 2007;29(8):1422–35.
3. Koehn P. Europarl: a parallel corpus for statistical machine translation. In Proc. 10th Machine Translation Summit, Phuket, Thailand, 2020, pp. 79–86.
4. Tan M, Pang R, Le QV. EfficientDet: scalable and efficient object detection. In Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Seattle, WA, USA, 2020, pp.10781–10790.
5. Viola P, Jones M. Rapid object detection using a boosted cascade of simple features. In Proc.2001 IEEE Computer Society Conf. Computer Vision and Pattern Recognition (CVPR 2001), 2001, pp.511–518.
6. Bengio Y, Bahdanau D, Cho K. Neural machine translation by jointly learning to align and translate. CoRR, vol. abs/1409.0473, 2014.
7. Sennrich R, Haddow B, Birch A. Neural machine translation of rare words with subword units. In Proc. 54th Annual Meeting of the Association for Computational Linguistics (ACL), Vancouver, Canada, 2020, pp. 1715–1725.
8. Eisenstein J. Introduction to natural language processing ,The MIT Press, 2019.
9. Brown TB, Mann B, Ryder N, Subbiah M, Kaplan J, Dhariwal P. GPT-3: language models are few-shot learners. arXiv preprint arXiv:2005.14165, 2020.
10. Microsoft. Microsoft/TROCR-base-handwritten. Hugging Face, 2021.
11. Smith R. An overview of the Tesseract OCR engine. In Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR) , Curitiba, Brazil, 2020, pp. 629–633.
12. Reese RM. Natural language processing with Java Cookbook: over 70 recipes to create linguistic and language translation applications using Java Libraries, Packt, 2019.
13. Abdurahimov M. Leveraging BERT for English-Arabic machine translation. ResearchGate, 2023.
14. Vaswani A et al. Attention is all you need. Advances in Neural Information Processing Systems (NeurIPS), 2017.
15. Bahdanau D, Cho K, Bengio Y. Neural machine translation by jointly learning to align and translate. arXiv preprint arXiv:1409.0473, 2014.
16. Aswani H. Practical applications of PyTesseract in data science. Medium, 2023.
17. Khan NS, Abid A, Abid K. A novel natural language processing (NLP)-based machine translation model for English to Pakistan sign language translation. Cogn Comput. 2020;12(4):748–65.
18. Escolano C, Costa-jussa MR, Fonollosa JA. From bilingual to multilingual neural machine translation by incremental training. In Proc. 2019 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2019, pp. 34–41.
19. Nguyen M, Lai V, Pouran Ben Veyseh A, Nguyen T. Trankit: a lightweight transformer-based toolkit for multilingual natural language processing. In Proc. 16th Conf. European Chapter of the Association for Computational Linguistics (EACL), 2021.
20. Devlin J, Chang MW, Lee K, Toutanova K. BERT: pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805. 2018.

21. Zhou X, Yao C, Wen H, Wang Y, Zhou S, He W, Liang L. EAST: an efficient and accurate scene text detector. In Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, USA, 2020, pp. 264-273.
22. Madill W. Exploring natural language processing (NLP). Localize Blog, 2022.
23. Qi P, Zhang Y, Zhang Y, Bolton J, Manning CD. Stanza: aPython natural language processing toolkit for many human languages. In Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL): System Demonstrations, 2020.
24. Paszke A, et al. PyTorch: an imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems (NeurIPS). Vancouver: Canada; 2020. p. 8024–35.
25. Abadi M, Barham P, Chen J, et al. TensorFlow: a system for large-scale machine learning. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI), pp. 265–283, 2023. Available: https://www.tensorflow.org.
26. Team K. Keras documentation: English-to-Spanish translation with a sequence-to-sequence transformer, 2021.
27. Sobel I, Feldman G. A 3x3 isotropic gradient operator for image processing. Technical Report: Stanford Artificial Intelligence Laboratory; 2020.
28. Ren K. Improving neural machine translation of Spanish to English. Stanford University, 2021.
29. Canny J. A computational approach to edge detection. IEEE Trans Pattern Anal Mach Intell. 1986;PAMI-8(6):679–698.
30. Pascanu D, Mikolov T, Bengio Y. On the difficulty of training recurrent neural networks. In Proc. 30th Int. Conf. Machine Learning (ICML), Atlanta, GA, USA, 2020, pp. 1310–1318.